

Homework 3

Due: April 12th at 11:59 pm

Deliverables. Submit a single PDF of your write-up to Gradescope *HW3 Write-Up*.

Start each problem on a new page.

Honor Code

Write and sign the following statement:

“I certify that all solutions in this document are entirely my own and that I have not looked at anyone else’s solution. I have given credit to all external sources I consulted.”

Signature: Jingliang Tu

1 Newton Might Have Been a Logistics Expert

- (a) Derive the gradient $\nabla_w J(w)$ of cost $J(w)$ as a matrix-vector expression. Also derive *all intermediate derivatives* in matrix-vector form. Do NOT specify them (**including the intermediates**) in terms of their individual components (e.g. w_i for vector w).

Solution:

Define the intermediate variable $z = Xw \in \mathbb{R}^n$. The logistic function $s = \sigma(z)$ is applied element-wise. Using differentials, we avoid component-wise derivatives. We know $\sigma'(z_i) = s_i(1 - s_i)$, so the differential ds is:

$$ds = (s \odot (\mathbf{1} - s)) \odot dz = \text{diag}(s \odot (\mathbf{1} - s))dz$$

For the intermediate variable z , since $z = Xw$, we have:

$$dz = Xdw$$

Now, compute the differential of the cost function $J(w)$ with respect to s :

$$dJ = \langle \nabla_s J, ds \rangle$$

where the gradient $\nabla_s J$ is:

$$\nabla_s J = -y \oslash s + (\mathbf{1} - y) \oslash (\mathbf{1} - s)$$

Here, $\oslash$ denotes element-wise division. Substituting ds into dJ :

$$\begin{aligned} dJ &= \langle -y \oslash s + (\mathbf{1} - y) \oslash (\mathbf{1} - s), s \odot (\mathbf{1} - s) \odot dz \rangle \\ &= \langle -y \odot (\mathbf{1} - s) + (\mathbf{1} - y) \odot s, dz \rangle \\ &= \langle s - y, dz \rangle \end{aligned}$$

This reveals the intermediate gradient with respect to z :

$$\nabla_z J = s - y$$

Finally, substituting $dz = Xdw$:

$$\begin{aligned} dJ &= \langle s - y, Xdw \rangle \\ &= \langle X^\top (s - y), dw \rangle \end{aligned}$$

Thus, the final gradient is:

$$\nabla_w J(w) = X^\top (s - y)$$

- (b) Derive the Hessian $\nabla_w^2 J(w)$ for the cost function $J(w)$ as a matrix-vector expression.

Solution:

The Hessian is the derivative of the gradient. Let $g = \nabla_w J(w) = X^\top (s - y)$. We take the differential of g :

$$\begin{aligned} dg &= X^\top d(s - y) \\ &= X^\top ds \end{aligned}$$

From part (a), we know $ds = \text{diag}(s \odot (\mathbf{1} - s))dz$ and $dz = Xdw$. Substituting these in:

$$dg = X^\top \text{diag}(s \odot (\mathbf{1} - s))Xdw$$

Let $S = \text{diag}(s \odot (\mathbf{1} - s))$. The differential dg relates to the Hessian H by $dg = Hdw$. Therefore, the Hessian is:

$$\nabla_w^2 J(w) = X^\top SX$$

- (c) Write the matrix-vector update law for one iteration of Newton's method, substituting the gradient and Hessian of $J(w)$.

Solution:

The standard update law for Newton's method is $w_{t+1} = w_t - [\nabla^2 J(w_t)]^{-1} \nabla J(w_t)$. Substituting the gradient and Hessian derived in parts (a) and (b), the update law is:

$$w_{t+1} = w_t - (X^\top SX)^{-1} X^\top (s - y)$$

where s and S are evaluated at w_t .

2 A Coordinated Leap of Faith

- (a) Show that stochastic gradient proposed in Equation ?? is a valid stochastic gradient, i.e. $\mathbb{E}_\xi[g(w; \xi)] = \nabla f(w)$ for every fixed $w \in \mathbb{R}^d$.

Solution:

By definition, $p = \text{Unif}(\{1, \dots, d\})$, meaning the probability of choosing any dimension i is $\frac{1}{d}$. The expectation is:

$$\begin{aligned} \mathbb{E}_\xi[g(w; \xi)] &= \sum_{i=1}^d p(\xi = i)g(w; i) \\ &= \sum_{i=1}^d \frac{1}{d} (d \langle \nabla f(w), e_i \rangle e_i) \\ &= \sum_{i=1}^d \frac{\partial f(w)}{\partial w_i} e_i \\ &= \nabla f(w) \end{aligned}$$

Thus, it is a valid stochastic gradient.

- (b) Suppose that f is twice differentiable... Show that for every $x, y \in \mathbb{R}^d$ satisfying $\text{supp}(x - y) \subseteq \{i\}$, the following quadratic upper bound is valid:

$$f(y) \leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{L_i}{2} \|y - x\|_2^2.$$

Solution:

Since $\text{supp}(x - y) \subseteq \{i\}$, x and y only differ along the i -th coordinate. We can write $y - x = \delta e_i$ for some scalar δ , which means $\delta^2 = \|y - x\|_2^2$. By Taylor's Theorem, there exists a point z on the line segment between x and y such that:

$$f(y) = f(x) + \langle \nabla f(x), y - x \rangle + \frac{1}{2} (y - x)^\top \nabla^2 f(z) (y - x)$$

Substitute $y - x = \delta e_i$ into the quadratic term:

$$(y - x)^\top \nabla^2 f(z) (y - x) = \delta e_i^\top \nabla^2 f(z) \delta e_i = \delta^2 (\nabla^2 f(z))_{ii}$$

By definition, $L_i = \sup_{w \in \mathbb{R}^d} (\nabla^2 f(w))_{ii}$, so $(\nabla^2 f(z))_{ii} \leq L_i$. Therefore:

$$\begin{aligned} f(y) &\leq f(x) + \langle \nabla f(x), y - x \rangle + \frac{1}{2} \delta^2 L_i \\ &= f(x) + \langle \nabla f(x), y - x \rangle + \frac{L_i}{2} \|y - x\|_2^2 \end{aligned}$$

- (c) First, we define $L_{\max} = \max_{1 \leq i \leq d} L_i$. Use the previous part to show that...

Solution:

At iteration t , we select coordinate i_t and update $w_{t+1} = w_t - \alpha d \nabla_{i_t} f(w_t) e_{i_t}$, where $\alpha = \frac{1}{L_{\max} d}$.

Note that $\text{supp}(w_{t+1} - w_t) \subseteq \{i_t\}$, so we can apply the upper bound from (b) using $L_{i_t} \leq L_{\max}$:

$$\begin{aligned} f(w_{t+1}) &\leq f(w_t) + \langle \nabla f(w_t), w_{t+1} - w_t \rangle + \frac{L_{\max}}{2} \|w_{t+1} - w_t\|_2^2 \\ &= f(w_t) - \alpha d (\nabla_{i_t} f(w_t))^2 + \frac{L_{\max}}{2} (\alpha d)^2 (\nabla_{i_t} f(w_t))^2 \\ &= f(w_t) - \left(\alpha d - \frac{L_{\max} \alpha^2 d^2}{2} \right) (\nabla_{i_t} f(w_t))^2 \end{aligned}$$

Substitute $\alpha = \frac{1}{L_{\max} d}$:

$$\alpha d - \frac{L_{\max} \alpha^2 d^2}{2} = \frac{1}{L_{\max}} - \frac{1}{2L_{\max}} = \frac{1}{2L_{\max}}$$

So, $f(w_{t+1}) \leq f(w_t) - \frac{1}{2L_{\max}} (\nabla_{i_t} f(w_t))^2$. Taking the expectation over $i_t \sim \text{Unif}(\{1, \dots, d\})$:

$$\mathbb{E}_{i_t}[(\nabla_{i_t} f(w_t))^2] = \sum_{i=1}^d \frac{1}{d} (\nabla_i f(w_t))^2 = \frac{1}{d} \|\nabla f(w_t)\|_2^2$$

This yields the first required result:

$$\mathbb{E}[f(w_{t+1})] \leq \mathbb{E} \left[f(w_t) - \frac{1}{2dL_{\max}} \|\nabla f(w_t)\|_2^2 \right]$$

Subtract $f_\star$ from both sides and apply the PL condition $\frac{1}{2} \|\nabla f(w_t)\|_2^2 \geq m(f(w_t) - f_\star)$:

$$\begin{aligned} \mathbb{E}[f(w_{t+1}) - f_\star] &\leq \mathbb{E}[f(w_t) - f_\star] - \frac{m}{dL_{\max}} \mathbb{E}[f(w_t) - f_\star] \\ &= \left(1 - \frac{m}{dL_{\max}} \right) \mathbb{E}[f(w_t) - f_\star] \end{aligned}$$

Unrolling this recursion t times gives the final convergence rate:

$$\mathbb{E}[f(w_t) - f_\star] \leq \left(1 - \frac{m}{dL_{\max}} \right)^t \mathbb{E}[f(w_0) - f_\star]$$

(d) Suppose now and for the remainder of the parts that $f(w) = \frac{1}{2} w^\top A w - w^\top b \dots$

Solution:

For the quadratic form, $\nabla f(w) = Aw - b$ and $\nabla^2 f(w) = A$. The coordinate descent update chooses dimension i and updates:

$$w_{t+1} = w_t - \alpha d (A_{i,:} w_t - b_i) e_i$$

where $A_{i,:}$ is the i -th row of A . The Hessian is constant (A), so its diagonal entries are A_{ii} . Thus, $L_{\max} = \max_i A_{ii}$. The PL constant m for a positive definite quadratic form is determined by its smallest eigenvalue, so $m = \lambda_{\min}(A)$.

(e) Compare the computational complexity of one iteration of coordinate descent versus one iteration of (full) gradient descent.

Solution:

For Coordinate Descent, a single iteration only requires computing the i -th component of the gradient, $\nabla_i f(w_t) = A_{i,:}w_t - b_i$. This is a dot product between two d -dimensional vectors, taking $\mathcal{O}(d)$ operations. For (full) Gradient Descent, we must compute the entire gradient vector $\nabla f(w_t) = Aw_t - b$, which involves a matrix-vector multiplication, taking $\mathcal{O}(d^2)$ operations.

(f) Note that gradient descent with $\alpha_t = 1/L$ satisfies...

Solution:

First, to prove $L_{\max} \leq L$, note that $L_{\max} = \max_i A_{ii} = \max_i e_i^\top A e_i$. Since e_i is a standard basis vector with $\|e_i\|_2 = 1$, and by the variational characterization $L = \lambda_{\max}(A) = \max_{\|v\|_2=1} v^\top A v$, it trivially follows that $L_{\max} \leq L$.

To reach an ε suboptimal solution:

- **Gradient Descent:** Requires $t_{GD} = \mathcal{O}\left(\frac{L}{m} \log(1/\varepsilon)\right)$ iterations. Each takes $\mathcal{O}(d^2)$. Total complexity: $\mathcal{O}\left(d^2 \frac{L}{m} \log(1/\varepsilon)\right)$.
- **Coordinate Descent:** Requires $t_{CD} = \mathcal{O}\left(d \frac{L_{\max}}{m} \log(1/\varepsilon)\right)$ iterations. Each takes $\mathcal{O}(d)$. Total complexity: $\mathcal{O}\left(d^2 \frac{L_{\max}}{m} \log(1/\varepsilon)\right)$.

Since $L_{\max} \leq L$, Coordinate Descent matches or strictly improves upon the overall computational complexity of Gradient Descent.

3 Differentiating the Differentiator

- (a) We start at the end and work backward. Find:

Solution:

- (i) $\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$
- (ii) $\frac{\partial L}{\partial w_2} = (\hat{y} - y)h$ and $\frac{\partial L}{\partial b_2} = \hat{y} - y$
- (iii) $\frac{\partial L}{\partial h} = (\hat{y} - y)w_2$

- (b) Now continue propagating backward through the activation function and first layer.

Solution:

- (i) By the chain rule on the element-wise function $h = \sigma(z)$, the Jacobian $\partial h / \partial z$ is a diagonal matrix with entries $\sigma'(z_j)$. Thus, $\frac{\partial L}{\partial z_j} = \frac{\partial L}{\partial h_j} \sigma'(z_j)$. In vector form, this is exactly $\frac{\partial L}{\partial z} = \frac{\partial L}{\partial h} \odot \sigma'(z)$.
- (ii) Since $z = W_1 x + b_1$, applying the matrix calculus identities given in Problem 1:

$$\frac{\partial L}{\partial W_1} = \frac{\partial L}{\partial z} x^\top \quad \text{and} \quad \frac{\partial L}{\partial b_1} = \frac{\partial L}{\partial z}$$

- (c) Suppose the network has $d = 1000$ input dimensions... How many such forward passes would you need? Why is the backward approach (backpropagation) dramatically more efficient...

Solution:

In the forward approach, we would need one independent forward pass for *every single parameter* to compute its effect on the scalar L . The total number of parameters is $(500 \times 1000) + 500 + 500 + 1 = 501,001$. Therefore, we would need **501,001** forward passes. Backpropagation is dramatically more efficient because L is a single scalar. Reverse-mode allows us to compute and share the intermediate gradient signals (like $\partial L / \partial z$) for all upstream nodes. A **single** backward pass evaluates the VJPs sequentially, accumulating gradients for all 501,001 parameters simultaneously.

- (d) Consider computing J by chaining together the individual Jacobians.

Solution:

- (i) To compute the full gradient $J \in \mathbb{R}^{1 \times n}$ in forward mode, we must run the entire K -step chain for each of the n standard basis vectors. Since there are K functions per chain, and n basis vectors, the total cost is n **complete forward passes (or $n \times K$ JVP evaluations)**.
- (ii) In reverse mode, starting with the scalar $v = 1$, we sequentially multiply on the left. We only need **1 complete backward pass (or K VJP evaluations)** to obtain the full gradient vector.

- (iii) Reverse mode is more efficient by a factor of n (which is 10^6 in this case). This directly explains the phenomenon in part (c), where computing gradients for 501,001 parameters requires just 1 backward pass instead of 501,001 forward passes.
- (e) In backpropagation for the one-hidden-layer network from parts (a)–(b), identify the sequence of VJPs that were computed.

Solution:

- **Stage 1 (Loss):** $v = 1$, $f_K(\hat{y}) = \frac{1}{2}(\hat{y} - y)^2$. The VJP produces $\frac{\partial L}{\partial \hat{y}} = \hat{y} - y$.
 - **Stage 2 (Layer 2):** $v = \frac{\partial L}{\partial \hat{y}}$, $f_{K-1}(h) = w_2^\top h + b_2$. The VJP produces the upstream gradient for the next layer $\frac{\partial L}{\partial h} = v \cdot w_2$ and local gradients for the parameters.
 - **Stage 3 (Layer 1 & Activation):** $v = \frac{\partial L}{\partial h}$, $f_1(x) = \sigma(W_1 x + b_1)$. The VJP passes through the element-wise activation to get $\frac{\partial L}{\partial z} = v \odot \sigma'(z)$, and then produces $\frac{\partial L}{\partial x} = W_1^\top \frac{\partial L}{\partial z}$ alongside parameter gradients $\frac{\partial L}{\partial W_1}$ and $\frac{\partial L}{\partial b_1}$.
- (f) **Implicit diff in action.** Consider the ridge regression problem...

Solution:

- (i) Optimality condition: $F(x, \lambda) = \nabla_x f(x, \lambda) = A^\top (Ax - b) + \lambda x = 0$.
- (ii) $\partial_1 F = \nabla_x^2 f = A^\top A + \lambda I$.
 $\partial_2 F = \partial_\lambda \nabla_x f = x$.
 Using the implicit differentiation formula $J = -A^{-1}B$:

$$\frac{\partial x^*}{\partial \lambda} = -(\partial_1 F)^{-1} \partial_2 F = -(A^\top A + \lambda I)^{-1} x^*$$

- (iii) Closed-form solution $x^*(\lambda) = (A^\top A + \lambda I)^{-1} A^\top b$. Let $M(\lambda) = A^\top A + \lambda I$. Differentiating directly:

$$\begin{aligned} \frac{\partial x^*}{\partial \lambda} &= \frac{dM^{-1}}{d\lambda} A^\top b \\ &= -M^{-1} \frac{dM}{d\lambda} M^{-1} A^\top b \\ &= -(A^\top A + \lambda I)^{-1} (I) (A^\top A + \lambda I)^{-1} A^\top b \\ &= -(A^\top A + \lambda I)^{-1} x^* \end{aligned}$$

The results match perfectly.

- (g) **Why not just unroll?**

Solution:

- (i) Unrolling requires storing the intermediate vectors $x_t \in \mathbb{R}^d$ for all T steps to construct the computational graph for backpropagation. The memory cost is $\mathcal{O}(d \cdot T)$.
- (ii) Implicit differentiation only needs the final state x^* and iterative solver memory for $AJ = -B$, which only requires storing vectors of size d . The memory cost is $\mathcal{O}(d)$, which is **completely independent of T** .

- (iii) For $d = 10,000$ and $T = 1,000$: Unrolling requires storing 10^7 floats, whereas implicit differentiation only requires 10^4 floats (a $1000\times$ memory reduction). Implicit diff saves massive memory by discarding the optimization trajectory, at the computational cost of solving one $d \times d$ linear system at the very end.
- (h) **When does implicit differentiation break down?**

Solution:

- (i) Example: $f(x, \theta) = \frac{1}{4}x^4 + \theta x$. The minimizer satisfies $\nabla_x f = x^3 + \theta = 0$, so $x^*(\theta) = (-\theta)^{1/3}$. At $\theta = 0$, the minimum is $x^*(0) = 0$. The Hessian is $\nabla_x^2 f(0, 0) = 3(0)^2 = 0$, which is non-invertible. Geometrically, the minimum is extremely "flat" (zero curvature), causing the optimal point to shift infinitely fast with respect to small perturbations in θ (the derivative is undefined/infinite).
- (ii) A poorly conditioned A means it has eigenvalues very close to zero. When computing the Jacobian using $\hat{x}$, we effectively multiply the approximation error by A^{-1} . Since A^{-1} has enormous eigenvalues, it acts as an amplifier, blowing up the small numerical deviation $\|\hat{x} - x^*\|$ into a massive error in the estimated Jacobian. This is the algebraic equivalent of the geometric "flatness" seen in (i).

4 I Can't Believe It's Not Distortion!

- (a) Show that relative entropy satisfies the Gibbs' inequality...

Solution:

Using the definition of KL divergence:

$$D_{\text{KL}}(p||q) = \sum_x p(x) \log \frac{p(x)}{q(x)} = \mathbb{E}_p \left[-\log \frac{q(X)}{p(X)} \right]$$

Since $-\log(z)$ is a strictly convex function, we apply Jensen's inequality:

$$\mathbb{E}_p \left[-\log \frac{q(X)}{p(X)} \right] \geq -\log \left(\mathbb{E}_p \left[\frac{q(X)}{p(X)} \right] \right)$$

Evaluating the inner expectation:

$$\mathbb{E}_p \left[\frac{q(X)}{p(X)} \right] = \sum_x p(x) \frac{q(x)}{p(x)} = \sum_x q(x) = 1$$

Therefore, $D_{\text{KL}}(p||q) \geq -\log(1) = 0$. Equality holds if and only if the random variable $q(X)/p(X)$ is a constant almost everywhere, meaning $q(x) = p(x)$ for all x .

- (b) Assume X is a discrete random variable...

Solution:

- (i) Let $q(x) = 1/m$.

$$D_{\text{KL}}(p||q) = \sum_x p(x) \log(m \cdot p(x)) = \log m \sum_x p(x) + \sum_x p(x) \log p(x) = \log m - H(X)$$

By Gibbs' inequality, $D_{\text{KL}}(p||q) \geq 0$, hence $\log m - H(X) \geq 0 \implies H(X) \leq \log m$.

- (ii) Let $q(x) = 2^{-\ell(x)}/C$.

$$D_{\text{KL}}(p||q) = \sum_x p(x) \log \frac{p(x)}{2^{-\ell(x)}/C} = \sum_x p(x) \log p(x) + \sum_x p(x) \ell(x) + \log C \sum_x p(x)$$

This simplifies to $-H(X) + \mathbb{E}_p[\ell(X)] + \log C \geq 0$. By Kraft's inequality, $C = \sum_x 2^{-\ell(x)} \leq 1$, which implies $\log C \leq 0$. Therefore:

$$-H(X) + \mathbb{E}_p[\ell(X)] \geq -\log C \geq 0 \implies \mathbb{E}_p[\ell(X)] \geq H(X)$$

- (c) Consider two candidate partitions... Which partition retains more information about Y ...

Solution:

For **Partition A** ($a = \{0, 1\}, b = \{2, 3\}$): $p(a) = 0.6$, $p(y = 1|a) \approx 0.0833$. $p(b) = 0.4$, $p(y = 1|b) \approx 0.8375$. The mutual information is $I_A(\tilde{X}; Y) \approx 0.462$ bits.

For **Partition B** ($a = \{0, 2\}, b = \{1, 3\}$): $p(a) = 0.7$, $p(y = 1|a) \approx 0.371$. $p(b) = 0.3$, $p(y = 1|b) \approx 0.416$. The mutual information is $I_B(\tilde{X}; Y) \approx 0.002$ bits.

Partition A retains vastly more information about Y . Intuitively, Partition A separates the inputs based on disease risk (low risk vs. high risk), successfully bottlenecking the critical diagnostic features. Partition B groups by arbitrary numeric parity, destroying almost all clinically relevant signals.

- (d) Now we will see that the information bottleneck automatically discovers the right notion of distortion...

Solution:

- (i) Start with $I(X; Y) - I(\tilde{X}; Y) = H(Y|\tilde{X}) - H(Y|X)$. Expand $H(Y|\tilde{X})$:

$$H(Y|\tilde{X}) = - \sum_{\tilde{x}} p(\tilde{x}) \sum_y p(y|\tilde{x}) \log p(y|\tilde{x})$$

Since $p(\tilde{x})p(y|\tilde{x}) = \sum_{x \in \tilde{x}} p(x)p(y|x)$, substitute this into the outer sum:

$$H(Y|\tilde{X}) = - \sum_x p(x) \sum_y p(y|x) \log p(y|\tilde{x}(x))$$

Subtract $H(Y|X) = - \sum_x p(x) \sum_y p(y|x) \log p(y|x)$:

$$\begin{aligned} I(X; Y) - I(\tilde{X}; Y) &= \sum_x p(x) \sum_y p(y|x) \log \frac{p(y|x)}{p(y|\tilde{x}(x))} \\ &= \sum_x p(x) D_{\text{KL}}(p(y|x) || p(y|\tilde{x}(x))) \\ &= \mathbb{E}_p[D_{\text{KL}}(p(y|X) || p(y|\tilde{X}))] \end{aligned}$$

- (ii) Using Partition A, $I(X; Y) \approx 0.480$ bits, so $I(X; Y) - I_A(\tilde{X}; Y) \approx 0.480 - 0.462 = 0.018$ bits. Computing the expected KL divergence directly using $p(y|x)$ and $p(y|\tilde{x})$ yields identically ≈ 0.018 bits.
- (iii) **Partition A.** Because $I(X; Y)$ is constant, the partition with the larger $I(\tilde{X}; Y)$ (which is A) must yield a strictly smaller difference (expected KL divergence).
- (iv) The information bottleneck framework reveals that the KL divergence between the true posterior $p(y|x)$ and the compressed posterior $p(y|\tilde{x})$ organically acts as the distortion function. This is notable because instead of relying on a human to arbitrarily define a geometric distance metric (like Mean Squared Error), the algorithm autonomously discovers the topological distance that best measures the loss of discriminative predictive power for Y .
- (e) For a layer to actually compress X ... explain in one sentence what property the mapping $X \rightarrow T$ must have...

Solution:

To reduce mutual information and compress the input, the mapping $X \rightarrow T$ must be **non-invertible (many-to-one)**, a property primarily introduced by non-linear activation functions (like ReLU) or pooling layers in neural networks.

- (f) Consider a network with two hidden layers...

Solution:

- (i) **No, T_2 is not necessarily a sufficient statistic.** Counterexample: Let $T_1 = X$ be the identity mapping, making it trivially a sufficient statistic. Let $T_2 = g(T_1) = 0$ be a function that maps all inputs to a constant. Then $I(T_2; Y) = 0 \neq I(X; Y)$, completely destroying the sufficient statistic property.

- (ii) Adding layers strips away irrelevant variance and noise from the input data that do not contribute to predicting Y . The benefit of depth is the iterative "purification" and untangling of representations, allowing the final layers to establish simple, linearly separable decision boundaries over a compressed, robust manifold.
- (g) Let's explore *why* adding noise to a representation can compress it...

Solution:

- (i) By the definition of mutual information: $I(X; T') = H(T') - H(T'|X)$. Since T' is a noisy version of T , conditioned on a specific $X = x$, T is fixed. Thus, the only uncertainty in T' comes from the noise E , where $P(E = 1) = \varepsilon$. Therefore, $H(T'|X) = H(E) = H(\varepsilon)$. Substituting this in gives: $I(X; T') = H(T') - H(\varepsilon)$.
- (ii) As ε increases to $1/2$, $I(X; T')$ approaches 0. When $\varepsilon = 1/2$, the noise flips bits completely uniformly, meaning T' becomes uniformly distributed noise regardless of X . Thus $H(T') \rightarrow 1$ and $H(\varepsilon) \rightarrow 1$, making $I(X; T') = 0$. This makes intuitive sense: maximum noise completely severs the informational link, destroying all knowledge of X .
- (iii) Irrelevant information (like precise spatial textures or background noise) is typically encoded in highly sensitive, fine-grained weight distributions, making it fragile and easily wiped out by gradient noise. In contrast, relevant information (like primary topological structures or macroscopic class boundaries) is encoded in robust, coarse-grained network states; small random walks in the parameter space during Phase 2 act as a regularization mechanism that naturally erodes the fragile overfitted details while preserving the robust, predictive core.